SourceSink



User Manual

Building, running and analyzing stock-and-flow models

This manual teaches system dynamics modeling with SourceSink through worked tutorials: building a model from scratch, exploring the example library, calibrating a model to field data, sensitivity and uncertainty analysis, and building models in conversation with the modeling assistant.

Version 0.2

October 2026

https://github.com/TNAziz/sourcesink

Contents

How to use this manual iii

1 Introduction 1
- 1.1 What SourceSink is . . . 1
- 1.2 Stocks and flows in one equation . . . 1
- 1.3 The building blocks . . . 2
- 1.4 Installing SourceSink . . . 2
- 1.5 A tour of the window . . . 2

2 Tutorial 1: Build your first model 4
- 2.1 Add the stock . . . 4
- 2.2 Add the flows . . . 5
- 2.3 Add converters and connectors . . . 6
- 2.4 Check the units . . . 7
- 2.5 Run the model and read the results . . . 8
- 2.6 Save your work . . . 9

3 Tutorial 2: Explore the example library 11
- 3.1 Ask “what if” with scenarios . . . 12
- 3.2 Read a model in plain language . . . 13

4 Tutorial 3: Calibrate a model to field data 15
- 4.1 Load the observations . . . 15
- 4.2 Interpret the fit . . . 16

5 Tutorial 4: Sensitivity and uncertainty 19
- 5.1 Screening with Morris . . . 19
- 5.2 Apportioning variance with Sobol indices . . . 19
- 5.3 Uncertainty bands . . . 21

6 Tutorial 5: Build a model with the assistant 22
- 6.1 Setting it up . . . 22
- 6.2 A conversation . . . 22
- 6.3 Answers grounded in runs . . . 23

7 Tutorial 6: Tidy a crowded diagram **25**
7.1 Opening legacy models 25

8 Tutorial 7: The command line **27**

9 Reference **28**
9.1 Elements and their options 28
9.2 Integration methods 28
9.3 Analysis methods at a glance 29
9.4 Themes 29

A Equation functions **30**

B Units **32**

C Keyboard shortcuts **33**

D Troubleshooting **34**

References **35**

How to use this manual

If you are new to system dynamics, read Chapter 1 and then work through Tutorial 1 (Chapter 2) with the app open beside you. It takes about twenty minutes and touches every basic tool. The other tutorials can be done in any order; each starts from a model that ships with SourceSink, so you never need to finish one tutorial to start another.

> **Conventions**
>
> Buttons and labels you click appear as **Run** or **Add parameter**. Keys appear as S or Ctrl+Enter. Model element names appear in code type, such as `P_mass`. Boxes like this one hold notes; blue boxes hold the steps of a tutorial; white boxes with a blue title hold the mathematics behind a feature. You can skip the mathematics on a first reading and still use every feature.

Chapter 9 is a reference for the editor, and the appendices collect the equation functions, unit syntax and keyboard shortcuts. A separate guide for AI agents and scripts, `docs/llm/cli-guide.md`, documents the command line and the MCP server in full.

CHAPTER 1

Introduction

1.1 What SourceSink is

SourceSink is a free, open-source tool for system dynamics: the practice of describing a system as quantities that accumulate (*stocks*) and the rates that change them (*flows*), wiring them together with the rules that connect them (*converters* and *connectors*), and then simulating forward in time to see what the structure does. Phosphorus in a lake, adopters of a new technology, carbon in the atmosphere, fish in a fishery and work in a project all fit this pattern.

SourceSink runs as a desktop application on Linux, Windows and macOS, in a web browser, and from the command line. Besides drawing and running models, it can fit a model to observations, rank which parameters matter, show the range of outcomes under uncertainty, check units, describe a model in plain language, and build a model with you through a conversation with an AI assistant. It also opens legacy `.stm` and `.itm` model files.

1.2 Stocks and flows in one equation

Every stock-and-flow model rests on one idea. A stock S changes only through the flows attached to it:

Accumulation

$$\frac{dS}{dt} = \sum_i \text{inflow}_i(t) - \sum_j \text{outflow}_j(t), \qquad S(t) = S(t_0) + \int_{t_0}^{t} \Big(\sum_i \text{inflow}_i - \sum_j \text{outflow}_j\Big)\, d\tau. \tag{1.1}$$

Flows and converters are ordinary functions of the stocks, of time and of constants. The diagram is a picture of these equations: a pipe into a stock is a + term, a pipe out of it is a − term, and a dashed connector into a flow or converter means "this equation reads that value."

SourceSink solves equation (1.1) numerically. With a time step Δt (called DT), Euler's method updates each stock as

$$S_{n+1} = S_n + \Delta t\Big(\sum_i \text{inflow}_i(t_n) - \sum_j \text{outflow}_j(t_n)\Big), \tag{1.2}$$

which is simple and transparent but needs a small DT for accurate results. Section 9.2 describes the higher-order methods (Runge–Kutta 2 and 4, and an adaptive Runge–Kutta 4/5 called **Adaptive (RK45)**) for models that change quickly.

1.3 The building blocks

Element	What it represents	How it looks
stock	A quantity that accumulates: water, people, money, phosphorus. Its equation is its *initial value*.	A white card with a blue name band and its latest value.
flow	A rate that fills or drains a stock: kg/year, people/day.	A small chip with a bow-tie mark sitting on a gray pipe.
converter	A constant or a calculation from other values.	A lavender pill; constants show their value.
connector	An information link: "this equation uses that value."	A dashed gray arrow.
cloud	The outside of the model: where a flow comes from or goes to when no stock is attached.	A faint cloud at the end of a pipe.

1.4 Installing SourceSink

You do not need to compile anything. Each release on the project's Releases page (`https://github.com/TNAziz/sourcesink/releases`) has installers:

System	File	Notes
Any Linux	one command (below)	Installs for your user only; no root needed.
Debian, Ubuntu	`SourceSink-…-amd64.deb`	`sudo apt install ./SourceSink-…-amd64.deb`
Arch, Omarchy	`SourceSink-…-x64.pacman`	`sudo pacman -U SourceSink-…-x64.pacman`
Windows	`SourceSink-…-x64.exe`	The installer is not yet signed; choose **Run anyway**.
macOS	`SourceSink-…-arm64.dmg`	Not yet signed: right-click the app and choose **Open** the first time.

On Linux, this command downloads the latest release and adds SourceSink to your application launcher:

```
curl -fsSL https://raw.githubusercontent.com/TNAziz/sourcesink/main/scripts/install.sh | sh
```

To run the newest code instead, install Node.js 22.19 or newer and run `npm install` and then `npm start` (browser) or `npm run desktop` (desktop window) in a copy of the repository.

1.5 A tour of the window

Figure 1.1 shows the main areas:

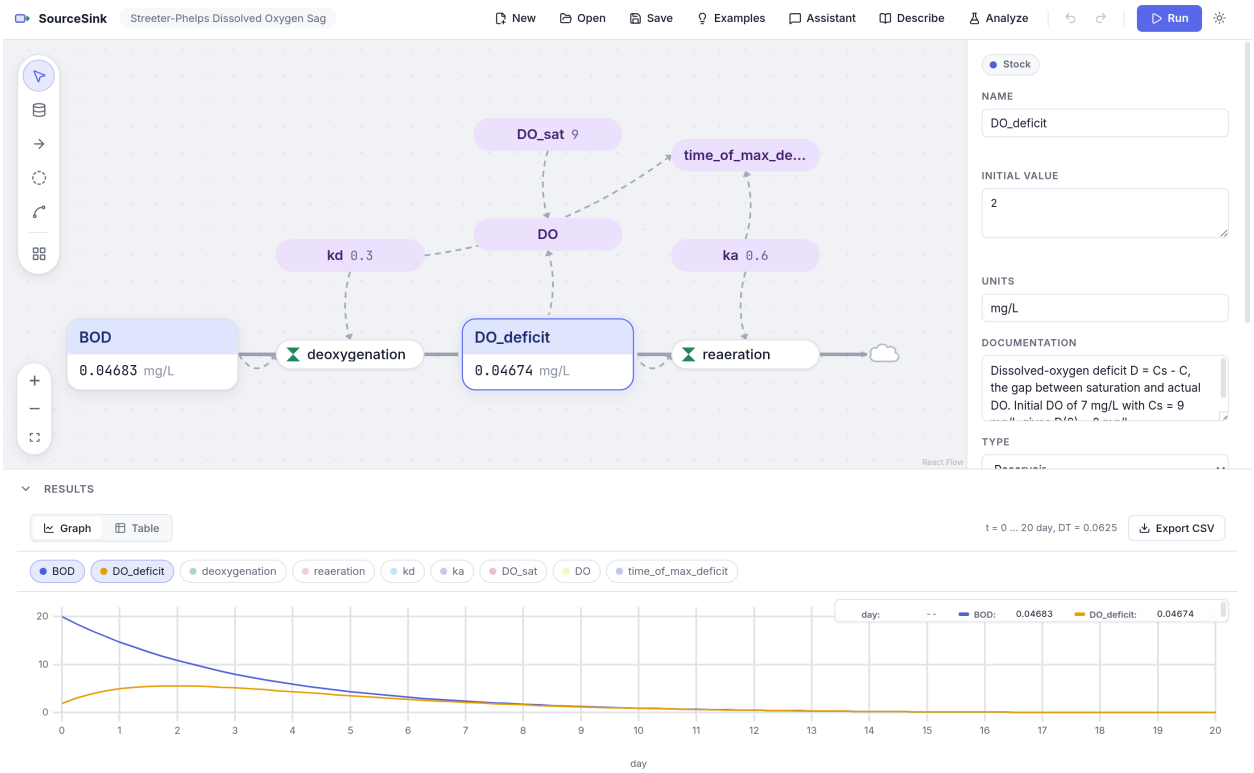


Figure 1.1: The SourceSink window with the Streeter–Phelps oxygen-sag example open and run. The toolbar is at the top, the tool palette floats on the left, the diagram fills the middle, the inspector on the right shows the selected stock, and the results drawer at the bottom plots the run.

Toolbar

New, **Open**, **Save**, the example gallery (**Examples**), the **Assistant**, **Describe** (a plain-language description of the model), **Analyze** (calibration, sensitivity and uncertainty), undo and redo, **Run**, and the light/dark theme switch.

Tool palette

Select (V), Stock (S), Flow (F), Converter (C), Connector (K), and **Tidy up** (L), which rearranges the diagram.

Canvas

The diagram. Drag to move elements, scroll to zoom, and use the zoom pill at the bottom left to fit the model to the window.

Inspector

The properties of whatever is selected: name, equation, units, documentation and element-specific options. With nothing selected it shows the model's run settings and scenarios.

Results drawer

Graph and table of the last run. Click a series name to show or hide it; **Export CSV** saves the numbers.

CHAPTER 2

Tutorial 1: Build your first model

In this tutorial you build a model of phosphorus in a lake from nothing. It is a classic environmental box model: a watershed delivers phosphorus to a lake, and the lake loses it two ways, by flushing out with the outflowing river and by settling into the sediment. You will learn to add each kind of element, write equations with units, run the model, read the results, and check the answer against the mathematics.

The lake as a box model

Let M be the phosphorus mass in the lake (kg), L the external load (kg/year), ρ the flushing rate and σ the settling rate (both per year). Then

$$\frac{dM}{dt} = \underbrace{L}_{\text{loading}} - \underbrace{\rho M}_{\text{flushing}} - \underbrace{\sigma M}_{\text{settling}} . \tag{2.1}$$

Starting from $M(0) = 0$, the exact solution is

$$M(t) = M^*(1 - e^{-t/\tau}), \qquad M^* = \frac{L}{\rho + \sigma}, \qquad \tau = \frac{1}{\rho + \sigma}. \tag{2.2}$$

With $L = 1000$ kg/year and $\rho = \sigma = 0.5$/year, the lake approaches $M^* = 1000$ kg with a response time $\tau = 1$ year. For a lake volume $V = 10^7$ m^3 the steady concentration is $C^* = M^*/V = 10^{-4}$ kg/m^3 $= 0.1$ g/m^3 $= 100$ µg/L, the classic result of Vollenweider's loading concept [6].

2.1 Add the stock

Step by step

1. Click **New** in the toolbar. The canvas is empty and shows a short hint (Figure 2.1).
2. Press S (or click the stock tool in the palette), then click on the canvas. A stock appears and the inspector shows its properties.
3. In the inspector, set **Name** to `P_mass`, **Initial value** to `0`, **Units** to `kg`, and **Documentation** to "Phosphorus dissolved in the lake."

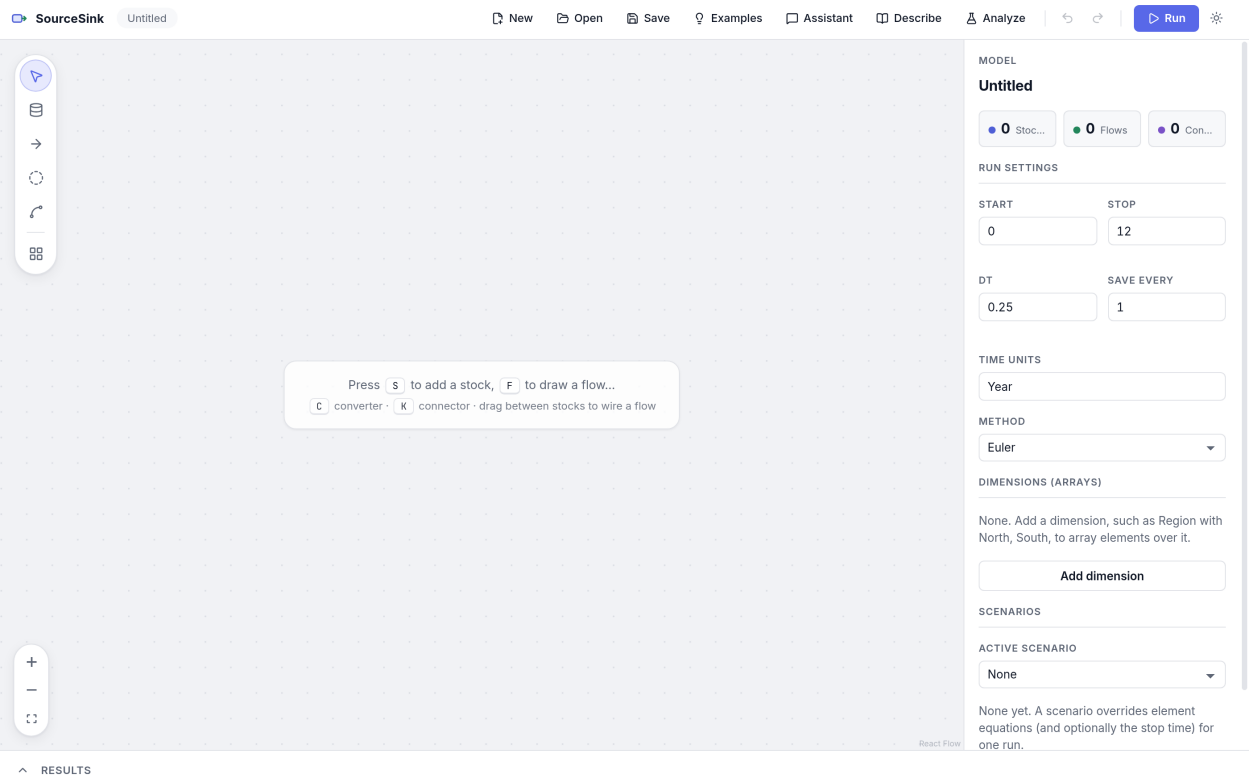


Figure 2.1: A new, empty model. The hint in the middle reminds you of the drawing keys.

> **Names**
>
> Names may contain letters, digits and underscores. Spaces are converted to underscores in equations, so `P mass` and `P_mass` refer to the same element. Short, readable names (`settling_rate`, not `sr`) make the model and its description easier to follow.

2.2 Add the flows

A flow is drawn by dragging. Dragging from empty canvas *into* a stock creates an inflow from a cloud; dragging *out of* a stock to empty canvas creates an outflow to a cloud; dragging from one stock to another creates a flow between them.

> **Step by step**
>
> 1. Press F. Drag from empty canvas on the left into `P_mass`. Name the new flow `loading`, set its equation to `1000` and its units to `kg/year`.
> 2. Drag from `P_mass` to empty canvas on the right. Name it `flushing`, units `kg/year`. Leave its equation for now.
> 3. Drag from `P_mass` downward to make a second outflow, `settling`, units `kg/year`.

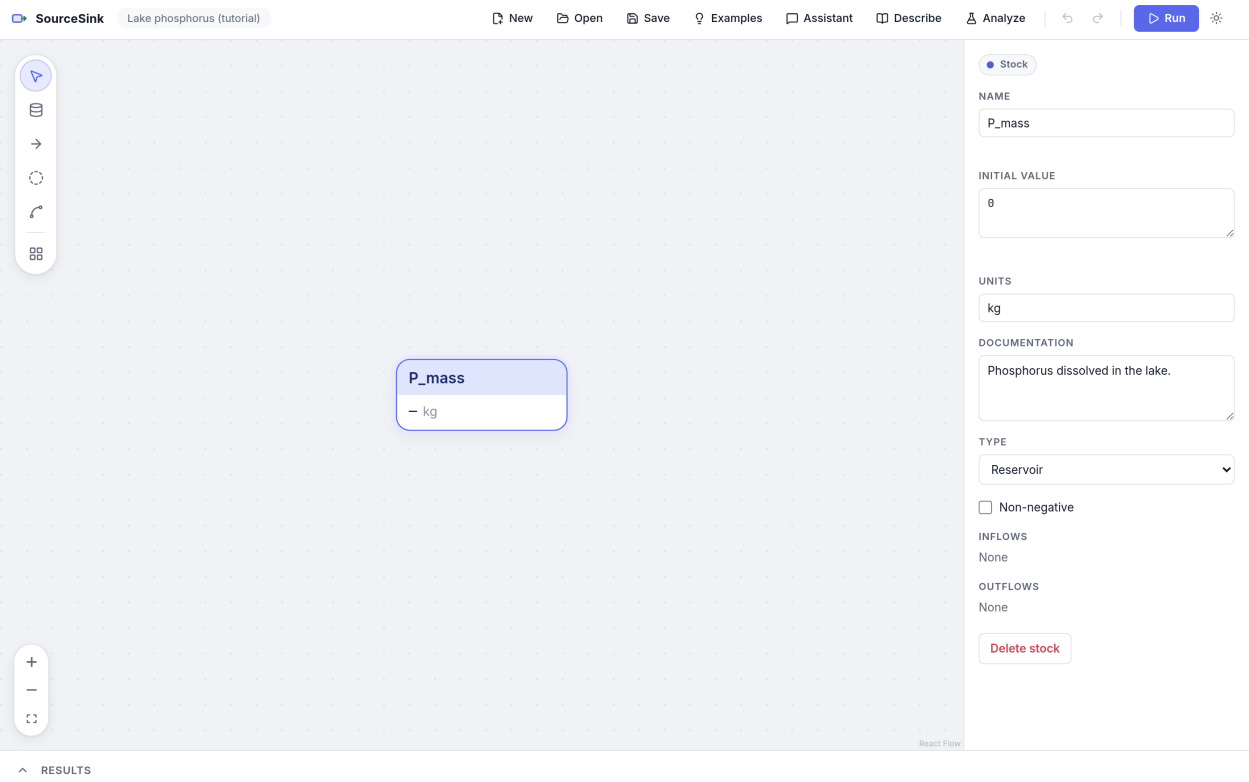


Figure 2.2: The stock `P_mass` after setting its name, initial value, units and documentation. A stock's equation is its *initial value*.

2.3 Add converters and connectors

The two outflows depend on rates. Converters hold those rates, and connectors tell each flow which values its equation may read.

Step by step

1. Press C and click near `flushing`. Name the converter `flushing_rate`, equation `0.5`, units `1/year`. Add a second converter `settling_rate` the same way.
2. Press K and drag from `flushing_rate` to `flushing`, then from `P_mass` to `flushing`. Do the same for `settling_rate` and `P_mass` into `settling`.
3. Select `flushing` and set its equation to `flushing_rate * P_mass`; set `settling` to `settling_rate * P_mass`. As you type, the inspector suggests names and functions; press Tab or Enter to accept one.
4. Add a converter `volume` (`1e7`, units `m3`) and a converter `concentration` with equation `P_mass / volume * 1000` and units `g/m3`, with connectors from `P_mass` and `volume`.

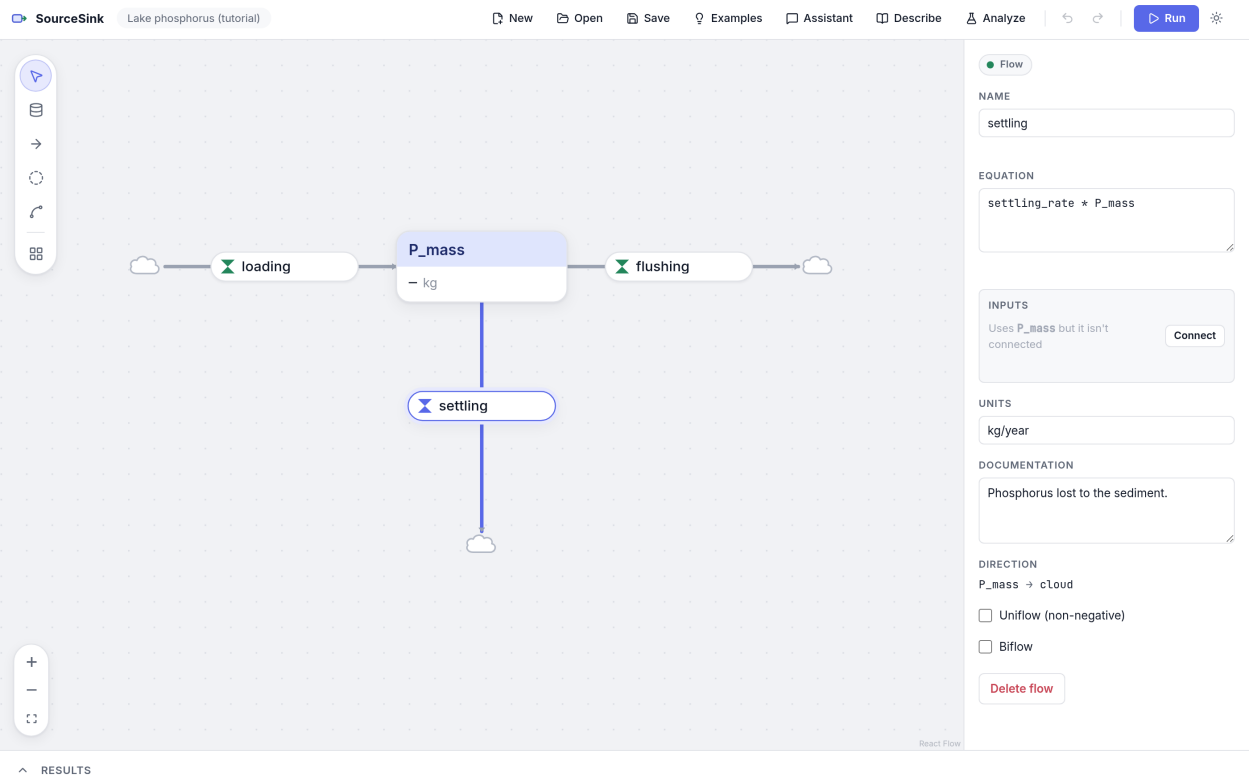


Figure 2.3: The stock with one inflow and two outflows. Each flow ends in a cloud: the watershed, the river downstream and the sediment are outside the model.

> **Why connectors matter**
>
> An equation should only use values that are connected to it. If you type a name that has no connector, the inspector says "Uses `P_mass` but it isn't connected" and offers a **Connect** button that draws the connector for you. This keeps the diagram an honest picture of the equations.

2.4 Check the units

SourceSink checks units as you work. A flow must be in the units of its stock per unit of model time, and both sides of a + or − must agree. If you had typed `kg` instead of `kg/year` for `flushing`, the inspector would explain the problem directly under the field (Figure 2.5).

How units are checked

Each unit is parsed into base dimensions with exponents, for example $\mathrm{mg/L} = \mathrm{mg}^{1}\,\mathrm{L}^{-1}$ and $1/\mathrm{day} = \mathrm{day}^{-1}$. The units of an expression follow from its operators:

$$[a \cdot b] = [a]\,[b], \qquad [a/b] = [a]/[b], \qquad [a \pm b] \text{ requires } [a] = [b], \qquad [a^{n}] = [a]^{n}.$$

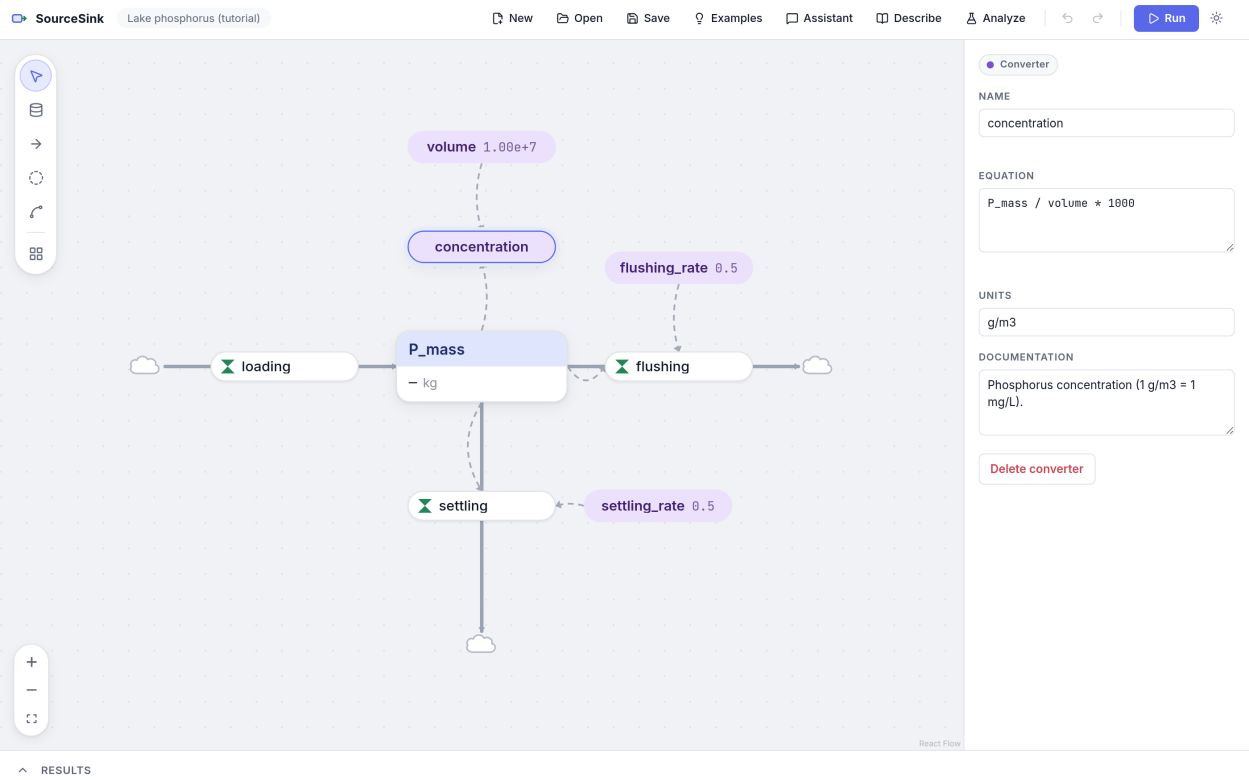


Figure 2.4: The finished lake model with the `concentration` converter selected. The factor 1000 converts kg/m^3 to g/m^3.

For a flow f attached to a stock S the check is $[f] = [S]/[t]$, where $[t]$ is the model's time unit. Scale differences within a dimension (mg and g, days and years) are reported with the factor that is missing, for example "the values differ by a factor of 365.25." Units are warnings, never errors: a model without units still runs.

2.5 Run the model and read the results

Step by step

1. Click anywhere on empty canvas to select nothing. The inspector now shows **Run settings**. Set **Start** 0, **Stop** 10, **DT** 0.05, **Save every** 0.1, **Time units** `year`, **Method** Euler.
2. Click **Run** (or press Ctrl+Enter). The results drawer opens with `P_mass` plotted, and each stock card shows its final value.
3. Click `concentration` in the series list to add it to the graph, and switch to **Table** to read exact numbers.

Check the result against equation (2.2). At $t = 1$ year the exact value is $1000\,(1 - e^{-1}) = 632$ kg; the table shows about 641 kg. The difference comes from Euler's method: with $\Delta t = 0.05$ each step

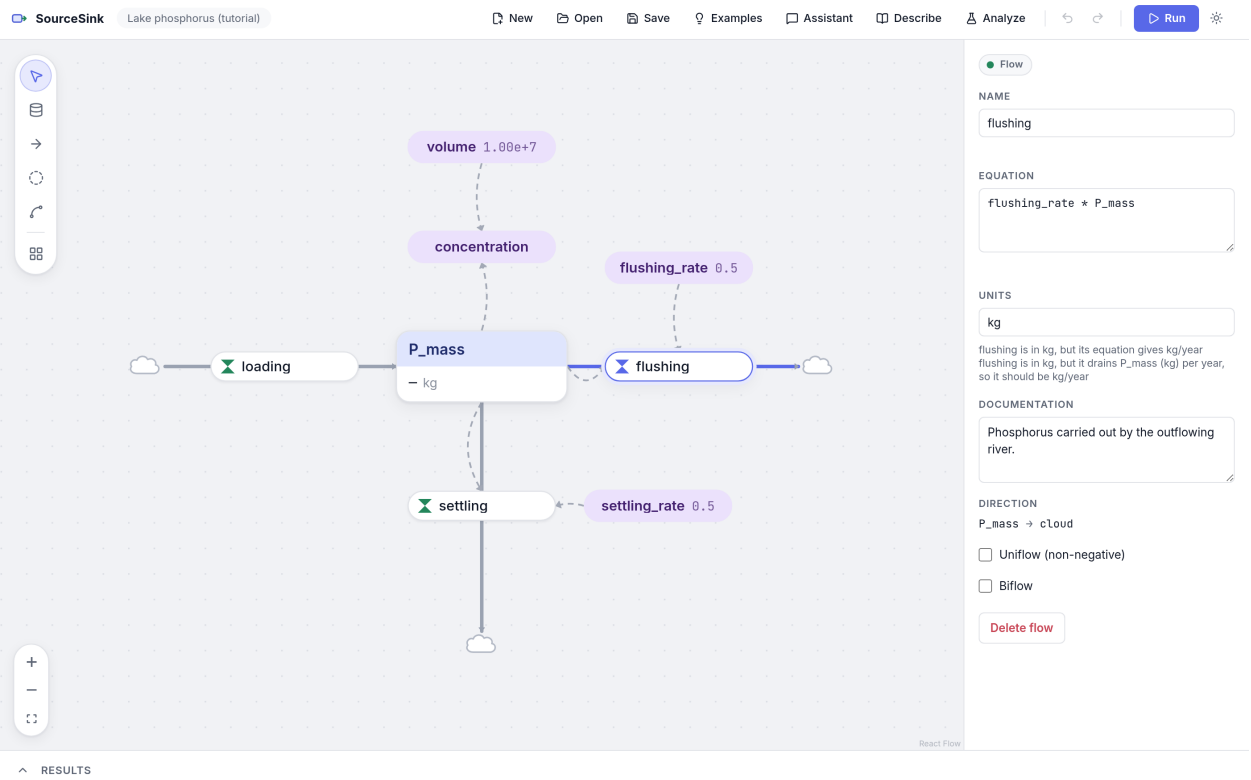


Figure 2.5: A unit mistake caught in the inspector: the flow `flushing` is declared in kg, but it drains a stock in kg per year.

loses a little accuracy, and the error shrinks in proportion to Δt.

> **Choosing DT**
>
> A reliable habit: run once, halve DT, and run again. If the results change by more than about one percent, the smaller DT (or a higher-order method) is needed. Fast processes, such as a rate of 7000/day acting on a stock of 250, need a DT much smaller than the fastest time constant; for those, choose the **Adaptive (RK45)** method, which picks its own step size.

2.6 Save your work

Click **Save** (Ctrl+S) to save the model as an XMILE file (`.xmile`), the open standard for system dynamics models. SourceSink also autosaves your work in the browser or desktop app, so an accidental close does not lose it.

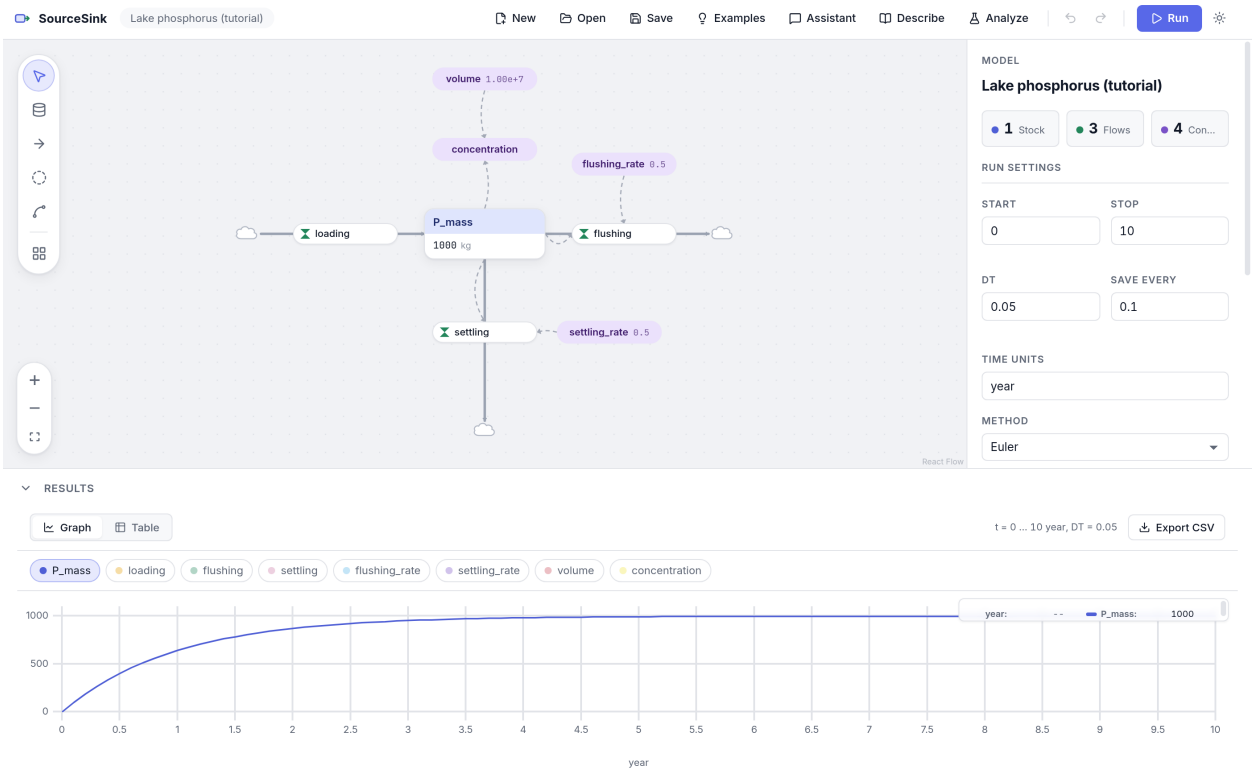


Figure 2.6: The lake fills toward its steady state of 1000 kg. The stock card shows the final value with its units.

CHAPTER 3

Tutorial 2: Explore the example library

SourceSink ships with 33 example models in six groups: environmental engineering, business and management, natural resources and economics, climate and the Earth system, populations and ecology, and short environmental problem-solving models in the spirit of Harte's *Consider a Spherical Cow* [2]. Every model cites its source and is tested against the published or analytic result.

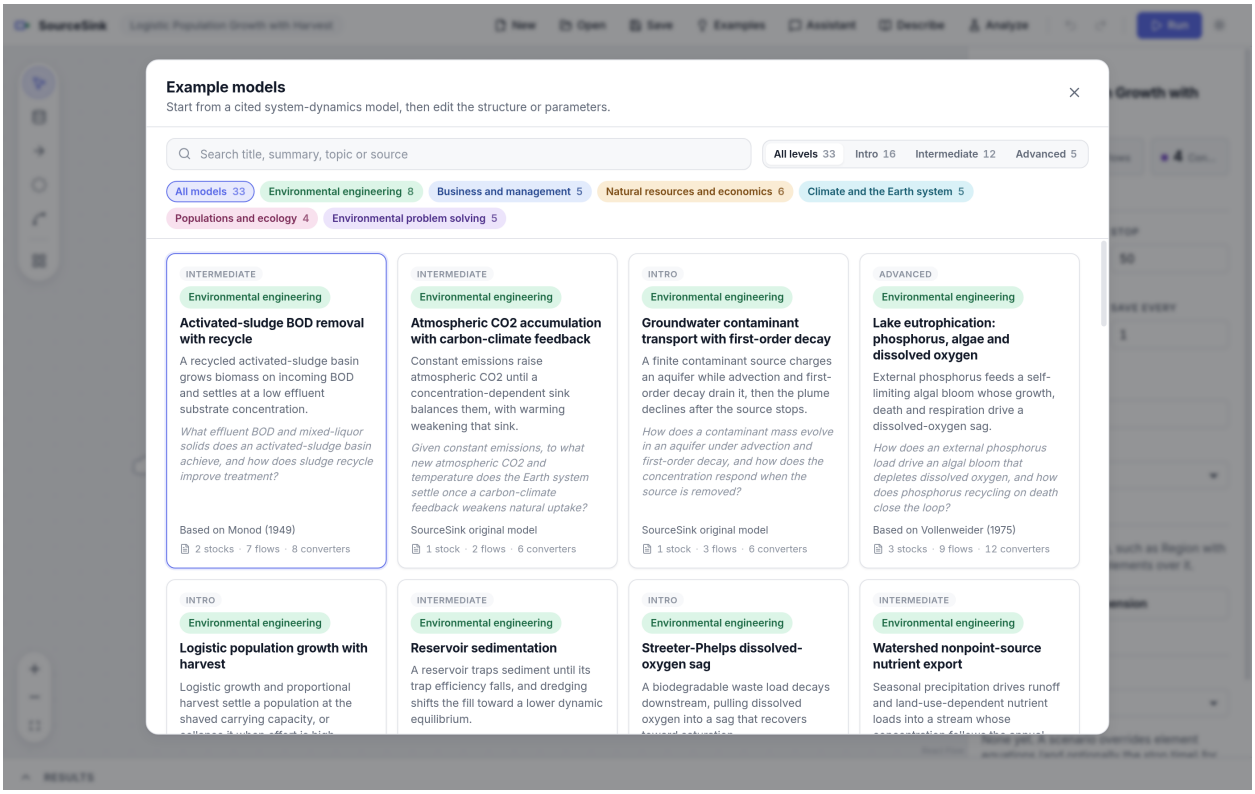


Figure 3.1: The example gallery: search, category filters with counts, a level filter, and one card per model with its source.

Open the Bass diffusion model

1. Click **Examples**. Choose **Business and management**.
2. Click the card **Bass diffusion**. The detail pane shows teaching notes, the question the model answers and the reference (Figure 3.2). Click a DOI to open the paper.
3. Click **Open model**, then **Run**.

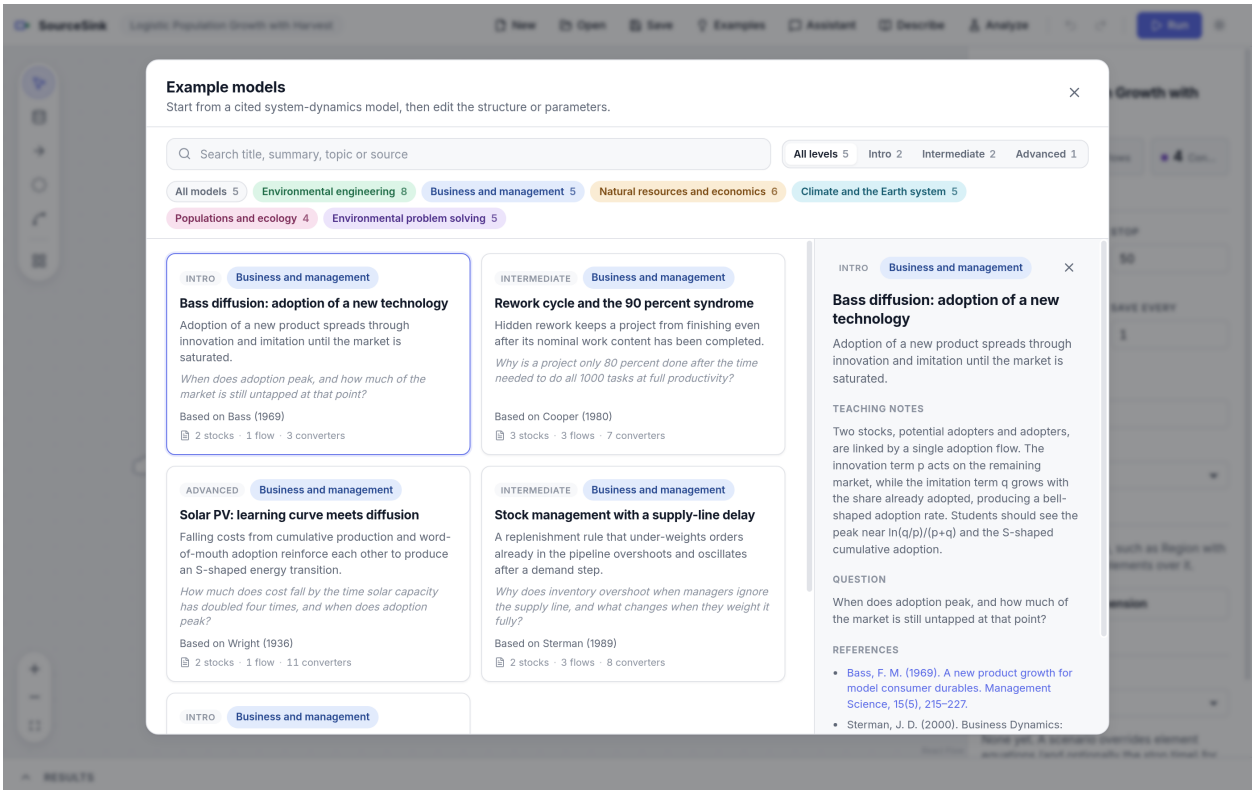


Figure 3.2: The detail pane for the Bass diffusion model, with teaching notes and references.

Bass diffusion

With N potential adopters in total and $A(t)$ adopters so far, adoption comes from innovators (coefficient p) and from imitation of existing adopters (coefficient q) [1]:

$$\frac{dA}{dt} = \left(p + q\,\frac{A}{N}\right)(N - A). \tag{3.1}$$

The adoption rate peaks at

$$t^* = \frac{\ln(q/p)}{p+q}, \tag{3.2}$$

so with $p = 0.03$ and $q = 0.38$ per year the peak comes at $t^* \approx 6.19$ years, with a peak rate of $N(p+q)^2/(4q) \approx 110{,}600$ adopters per year for $N = 10^6$.

3.1 Ask “what if” with scenarios

A scenario changes some equations for one run without editing the model.

Compare a stronger word-of-mouth effect

1 Click empty canvas so nothing is selected. In the inspector, find **Scenarios** and click **Add**

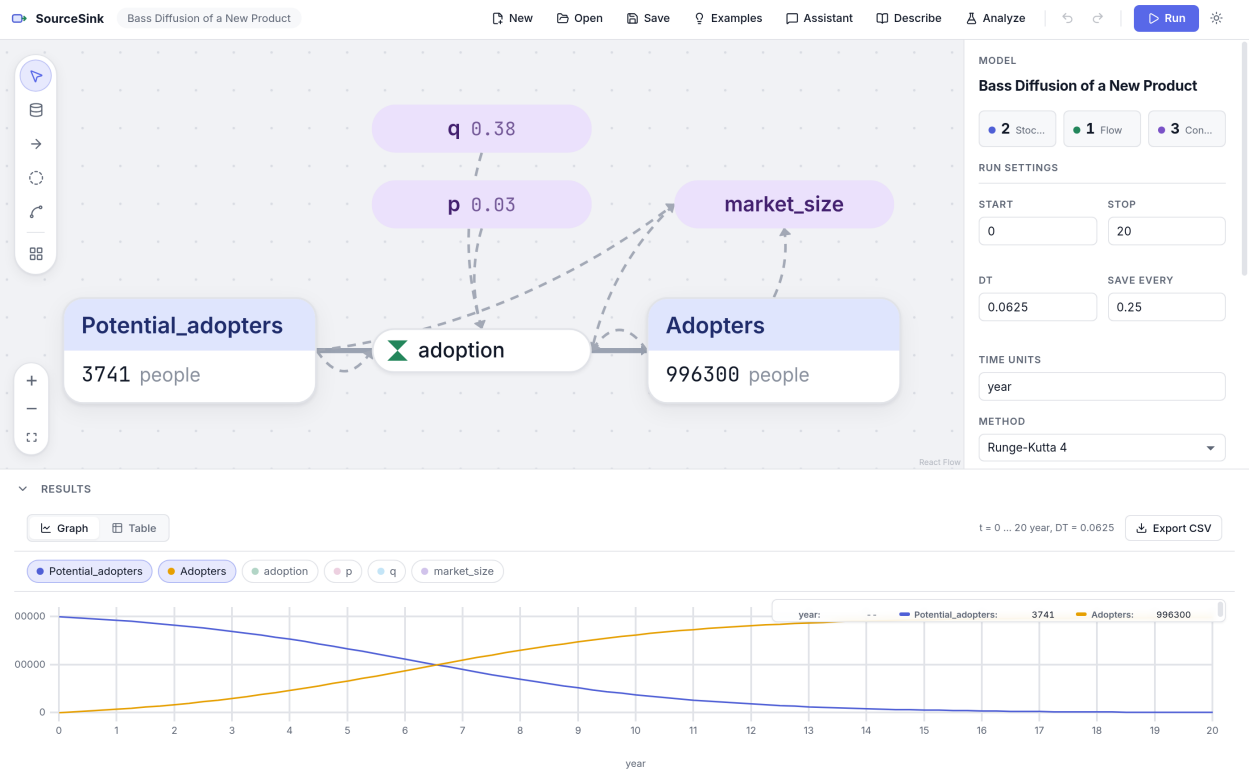


Figure 3.3: The Bass model after a run: the S-shaped growth of adopters.

scenario. Name it "Strong word of mouth."

2. Click **Add override**, choose the imitation coefficient and set it to `0.6`.
3. Choose the scenario under **Active scenario** and click **Run**. The peak moves earlier: $t^* = \ln(0.6/0.03)/0.63 \approx 4.76$ years.
4. Set **Active scenario** back to **None** to return to the base model.

3.2 Read a model in plain language

Click **Describe** to see a written description of the open model: every stock, what fills and drains it, every flow and converter with its equation, the feedback loops with their polarity (reinforcing R or balancing B), and the time settings. A coverage check confirms that every element and link is mentioned. It is a good way to explain a model to someone who did not build it, and to spot an element you forgot to connect.

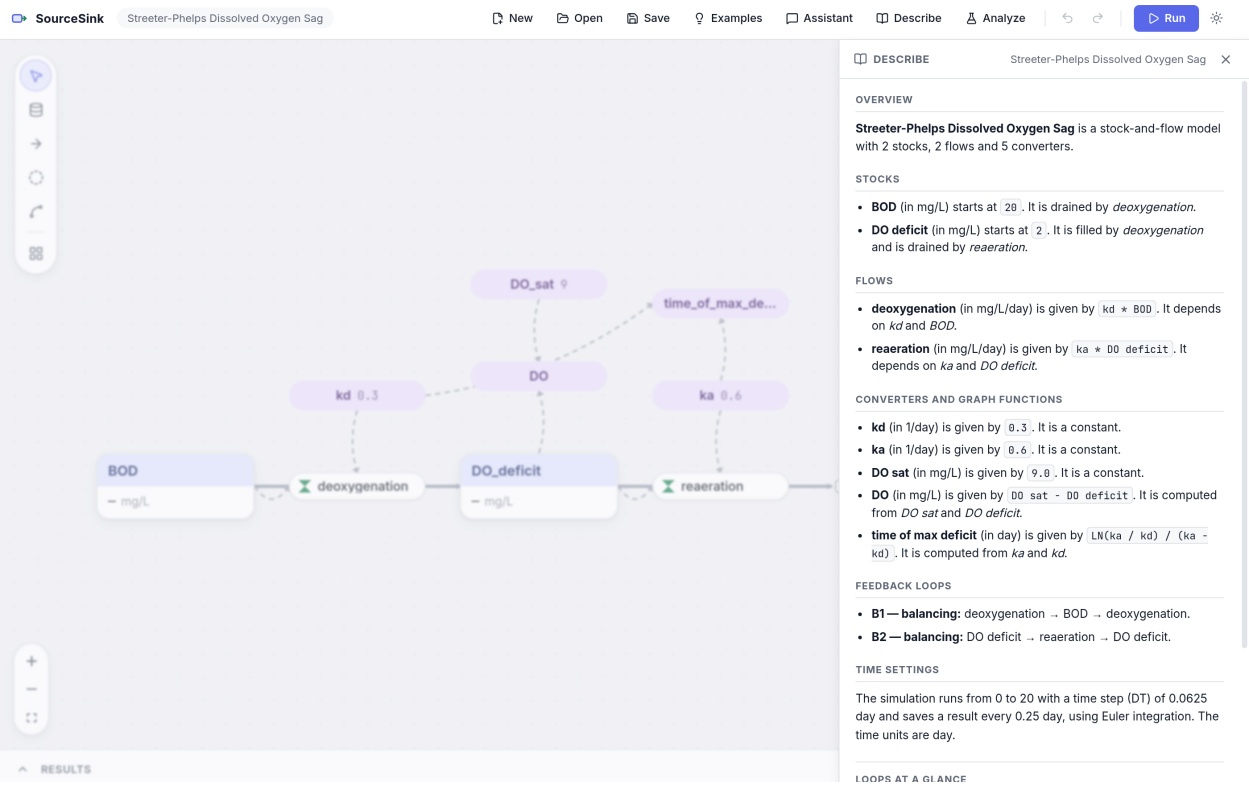


Figure 3.4: The Describe panel for the Streeter–Phelps model, listing both balancing feedback loops.

CHAPTER 4

Tutorial 3: Calibrate a model to field data

Hand-tuning parameters until a curve "looks right" is slow and hard to defend. Calibration finds the parameter values that best reproduce your observations and tells you how precisely the data pin them down. In this tutorial you fit the deoxygenation and reaeration rates of the Streeter–Phelps model [5] to a set of field measurements.

The Streeter–Phelps oxygen sag

Below a waste discharge, the remaining biochemical oxygen demand L (BOD) and the dissolved-oxygen deficit D evolve as

$$\frac{dL}{dt} = -k_d L, \qquad \frac{dD}{dt} = k_d L - k_a D, \tag{4.1}$$

where k_d is the deoxygenation rate and k_a the reaeration rate. The deficit peaks at the critical time

$$t_c = \frac{1}{k_a - k_d} \ln\left[\frac{k_a}{k_d}\left(1 - \frac{D_0\,(k_a - k_d)}{k_d L_0}\right)\right]. \tag{4.2}$$

4.1 Load the observations

The file `examples/environmental/streeter-phelps-observed.csv` holds 21 days of measurements of BOD and the oxygen deficit, with some missing values. Its column headers include units, such as `BOD (mg/L)`.

Step by step

1. Open the example **Streeter–Phelps dissolved-oxygen sag** and click **Analyze**. The **Calibrate** tab is open.
2. Drop the CSV file on the panel, or click **Load CSV…**. SourceSink reads the first numeric column as time and matches the other columns to model elements by name; a header like `BOD (mg/L)` matches `BOD`.
3. Under **Parameters**, choose `kd` with range 0.05 to 1, click **Add parameter** and choose `ka` with range 0.05 to 2. Tick **log** for parameters that span orders of magnitude.
4. Keep **Nelder–Mead** and **SSE**, and click **Fit**. The fit runs in the background and takes well under a second here.

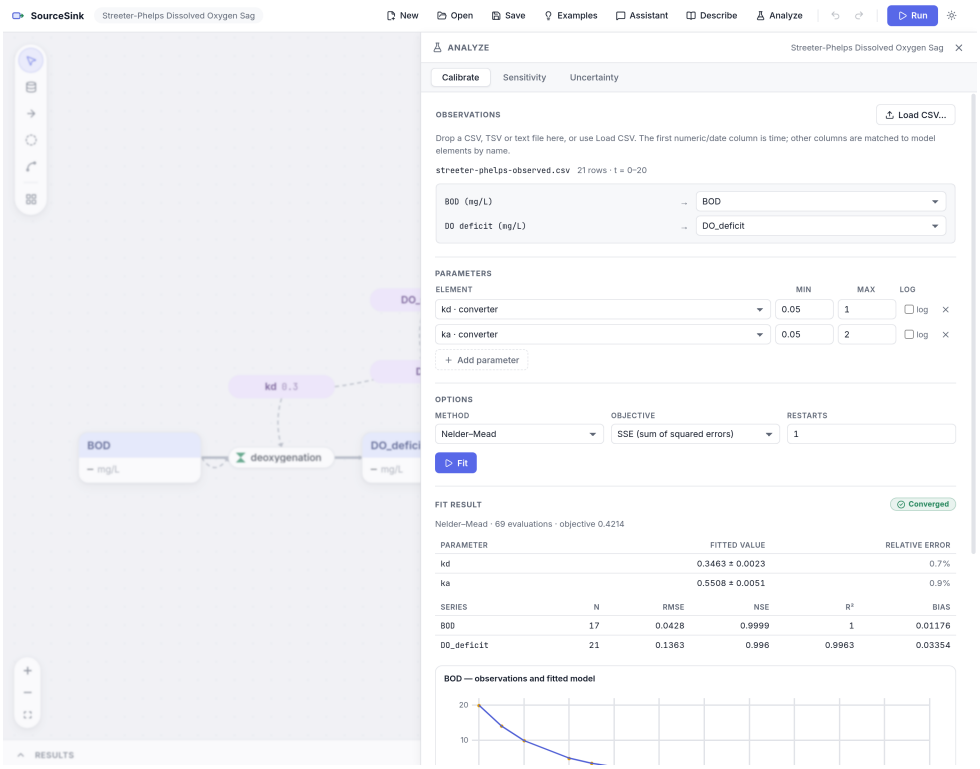


Figure 4.1: The calibration result: both rates are recovered with relative errors below one percent, and both series are fitted with a Nash–Sutcliffe efficiency above 0.99.

4.2 Interpret the fit

The data were generated with $k_d = 0.35$ and $k_a = 0.55$ per day plus noise. The fit returns $k_d = 0.3463 \pm 0.0023$ and $k_a = 0.5508 \pm 0.0051$.

Objective, goodness of fit and standard errors

For observations y_i and simulated values $\hat{y}_i(\theta)$ at the same times, the default objective is the sum of squared errors

$$\mathrm{SSE}(\theta) = \sum_i \left(y_i - \hat{y}_i(\theta)\right)^2. \tag{4.3}$$

With several series of different magnitude, the variance-weighted SSE divides each series' SSE by the variance of its observations. Each series is summarized by the RMSE, the bias, R^2 and the Nash–Sutcliffe efficiency

$$\mathrm{NSE} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \tag{4.4}$$

which is 1 for a perfect fit and 0 for a model no better than the mean. Standard errors come from the Jacobian J of the residuals at the optimum:

$$\mathrm{Cov}(\hat{\theta}) \approx s^2 \, (J^{\mathsf{T}} J)^{-1}, \qquad s^2 = \frac{\mathrm{SSE}(\hat{\theta})}{n - p}, \tag{4.5}$$

with n observations and p parameters. Strongly correlated parameter estimates (correlation near ± 1) mean the data cannot tell those parameters apart; SourceSink reports this as an identifiability warning.

Scroll down to see each series plotted against its observations (Figure 4.2). When you are satisfied, click **Apply fitted values** to write the estimates into the model as one undoable change.

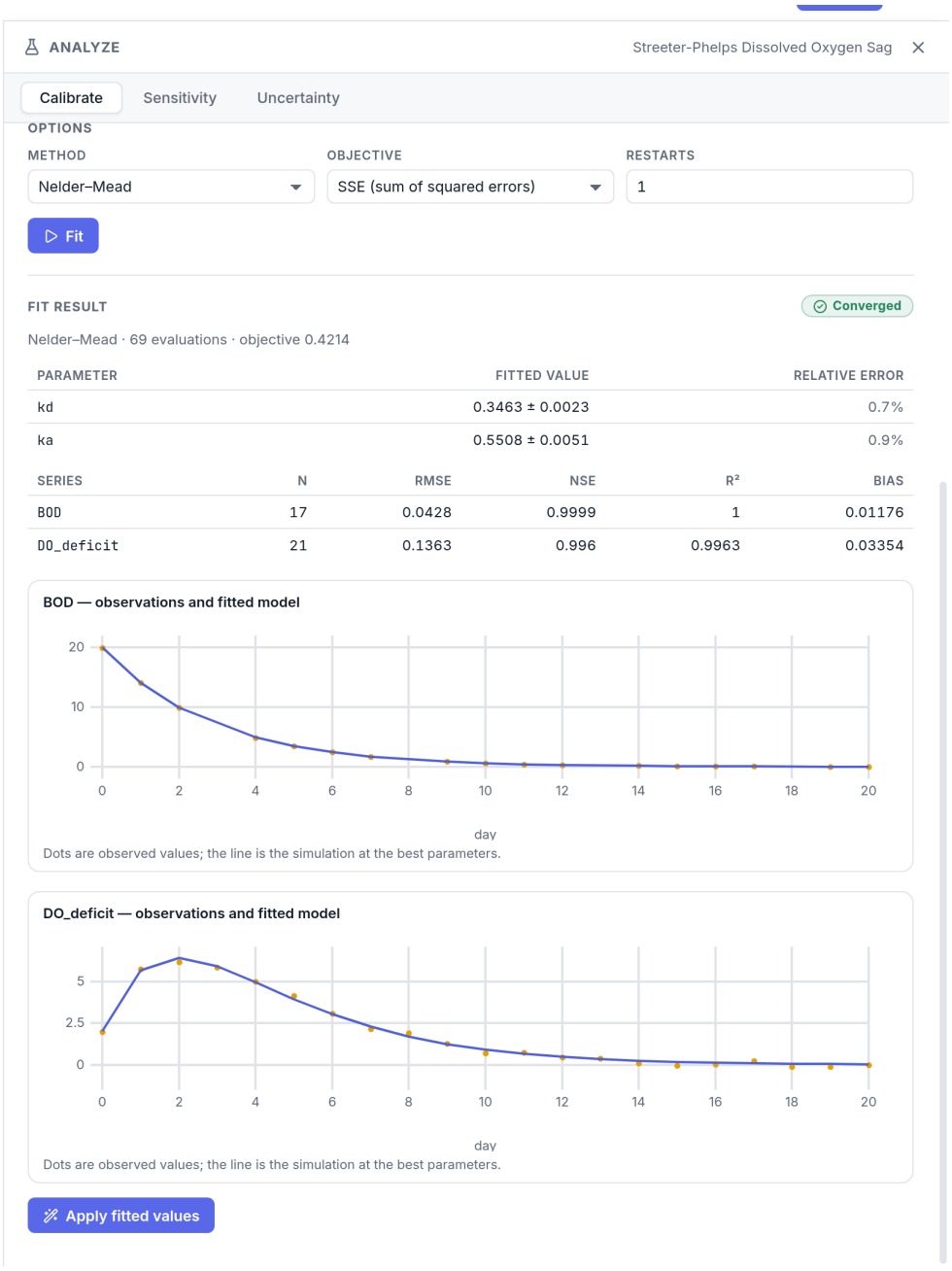


Figure 4.2: Observations (dots) and the fitted model (lines).

From the command line

The same fit runs without the app:

```
sourcesink fit examples/environmental/streeter-phelps.xmile \
  --data examples/environmental/streeter-phelps-observed.csv \
  --param kd=0.05:1 --param ka=0.05:2
```

CHAPTER 5

Tutorial 4: Sensitivity and uncertainty

Which parameters matter most for the answer you care about? How wide is the range of outcomes when the inputs are uncertain? This tutorial uses the **Sensitivity** and **Uncertainty** tabs of the **Analyze** panel on the Streeter–Phelps model, with the question: *what controls the lowest dissolved oxygen downstream?*

5.1 Screening with Morris

Step by step

1. With the Streeter–Phelps model open, click **Analyze** and choose the **Sensitivity** tab.
2. Under **Output metric**, choose series `DO` and statistic **Minimum**.
3. Add three parameters: `kd` 0.2–0.5, `ka` 0.4–0.8 and `DO_sat` 8–10.
4. Keep **Quick screen (Morris)** with 20 trajectories and click **Run**. The cost is $20 \times (3+1) = 80$ runs.

Elementary effects

Each parameter range is mapped to $[0, 1]$. Along random one-at-a-time trajectories, the elementary effect of parameter i is

$$EE_i = \frac{f(\mathbf{x} + \Delta\, \mathbf{e}_i) - f(\mathbf{x})}{\Delta}, \tag{5.1}$$

and over r trajectories Morris' method [3] reports

$$\mu_i^* = \frac{1}{r} \sum_{k=1}^{r} |EE_i^{(k)}|, \qquad \sigma_i = \mathrm{sd}(EE_i^{(k)}). \tag{5.2}$$

A large μ^* means the parameter matters; a large σ means its effect depends on the other parameters (non-linearity or interaction).

5.2 Apportioning variance with Sobol indices

Switch the method to **Variance-based (Sobol)** and set **Base samples** to 512 (cost $512 \times (3+2) = 2560$ runs). The result (Figure 5.1) shows, for each parameter, the share of the output variance it explains

alone (S_1) and including its interactions (S_T), with bootstrap confidence intervals.

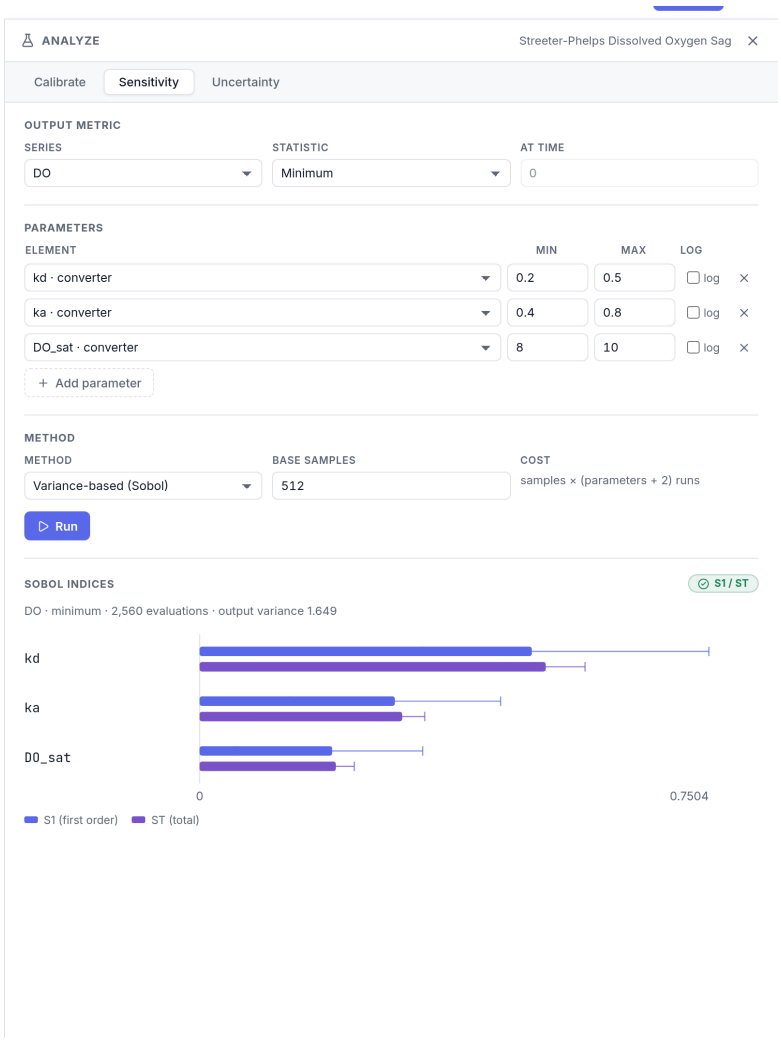


Figure 5.1: Sobol indices for the minimum dissolved oxygen. The deoxygenation rate explains the largest share, followed by the reaeration rate and the saturation concentration.

First-order and total indices

For an output $Y = f(X_1, \dots, X_k)$,

$$S_i = \frac{\mathrm{Var}[\mathbb{E}(Y \mid X_i)]}{\mathrm{Var}(Y)}, \qquad S_{T,i} = \frac{\mathbb{E}[\mathrm{Var}(Y \mid X_{\sim i})]}{\mathrm{Var}(Y)}, \tag{5.3}$$

where $X_{\sim i}$ is every input except X_i. SourceSink estimates them with the Saltelli [4] and Jansen estimators on a Sobol quasi-random sequence, with $N(k+2)$ model runs. $S_T - S_1$ measures how much a parameter acts through interactions; $\sum_i S_i < 1$ signals interactions in the model as a whole.

5.3 Uncertainty bands

The **Uncertainty** tab samples every parameter range with a Latin hypercube, runs the model for each sample (200 runs by default) and shows the median with 25–75% and 5–95% bands for the series you choose.

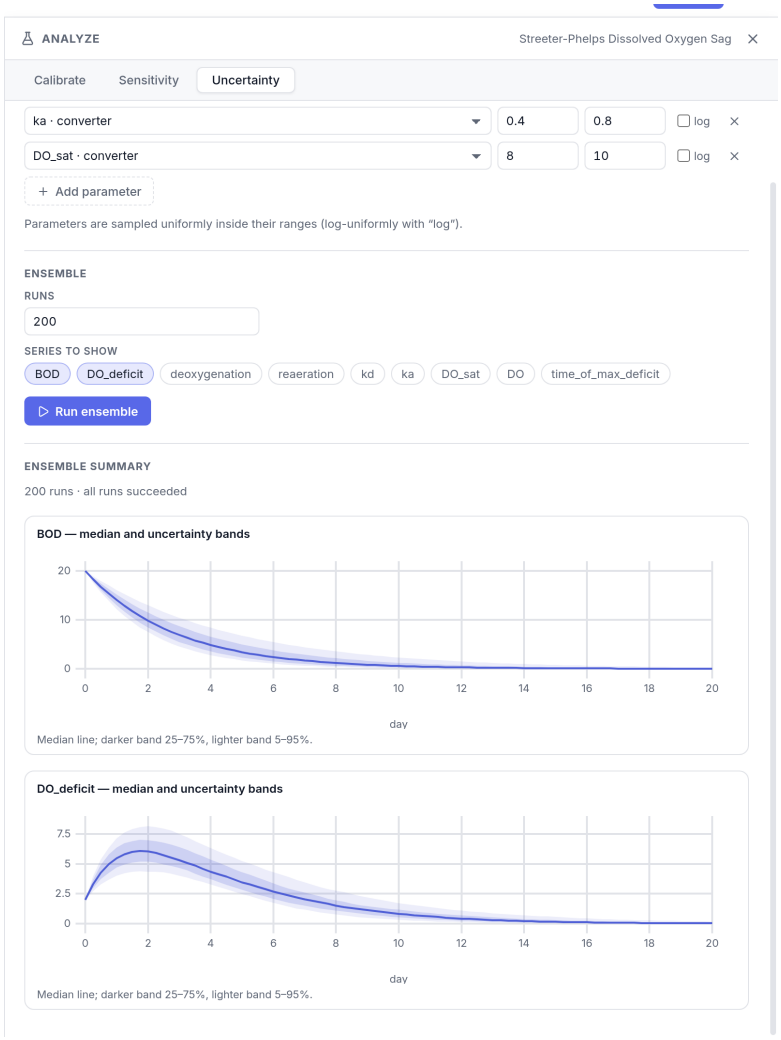


Figure 5.2: Uncertainty bands for BOD and the oxygen deficit: the median line with the 25–75% (darker) and 5–95% (lighter) bands.

Reading the bands

The bands describe the consequence of the ranges you entered, not the probability that the real river behaves this way. Narrow your ranges with data first (Tutorial 3); then the bands become a statement about what you actually know.

CHAPTER 6

Tutorial 5: Build a model with the assistant

The modeling assistant lets you describe a system in ordinary language. It asks follow-up questions, drafts the model as a set of proposed changes for you to review, and can run the model to answer questions from the results.

6.1 Setting it up

The assistant uses a language model through OpenCode (`https://opencode.ai`) and OpenRouter, so you choose the model and pay the provider directly; a typical turn with a fast model costs about a tenth of a cent. Install OpenCode and sign in to OpenRouter once with `opencode providers`. If the assistant cannot find OpenCode it says so and explains what to install. Everything else in SourceSink works without it.

6.2 A conversation

Step by step

1. Click **New**, then **Assistant**. Choose a model in the picker at the top of the panel.
2. Describe the system. For example: "A lake receiving phosphorus from farms. Lake phosphorus starts at 50 tonnes. Farm loading is 10 tonnes per year. Phosphorus leaves by outflow at 10% per year and settles to the sediment at 5% per year. Run 30 years."
3. The assistant answers, asks up to three follow-up questions, and shows its **Proposed changes** (Figure 6.1). Each added or changed element has a checkbox. Untick anything you do not want, then click **Apply selected**.

What keeps the assistant honest

The assistant cannot edit your model directly. It can only propose changes, and every proposal passes the same validation SourceSink applies to its own files: the equations parse, every referenced name exists and is connected, units are checked, and the model runs. A proposal that fails is sent back to the language model with the exact problem, up to three times. Only then is it shown to you, and nothing changes until you click **Apply selected**. Applied elements carry a small blue dot on the canvas marking them as AI-proposed.

The stage tracker at the top of the panel (**Question**, **Boundary**, **Stocks**, **Flows**, **Feedback**, **Numbers**, **Next steps**) shows where the interview is. You can answer its questions, skip ahead, or ask

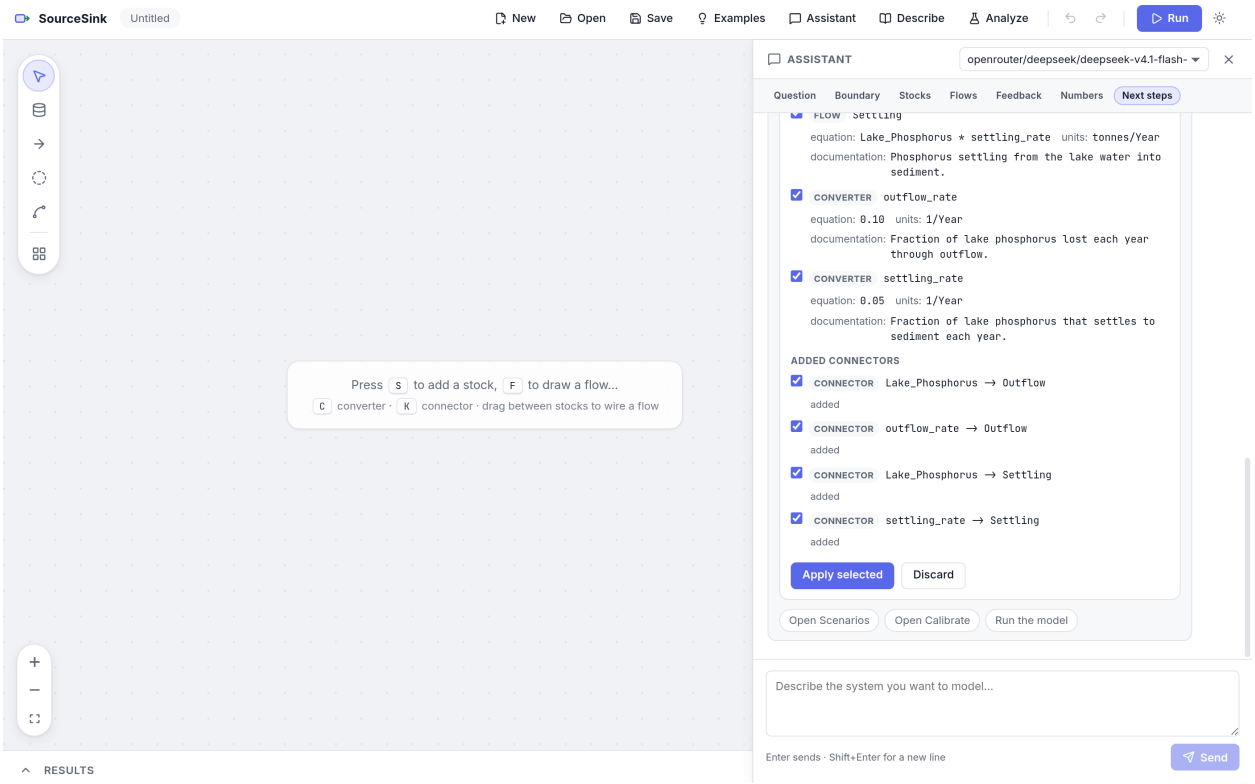


Figure 6.1: The end of the assistant's first reply: every proposed element and connector with a checkbox, **Apply selected** and **Discard**, and suggestion buttons for what to do next. Above it (scrolled out of view) are the assistant's summary, its follow-up questions and the cost of the turn.

for something else at any point.

6.3 Answers grounded in runs

Ask a question about behavior, such as "What happens to lake phosphorus if farm loading is cut in half? When does it fall below 45 tonnes?" The assistant asks SourceSink to run the current model and a what-if variant, reads the results, and answers from them. Each run is shown as a card with the facts it produced and a small chart (Figure 6.2).

Checking the assistant's numbers

The lake obeys $dP/dt = L - (0.10 + 0.05)P$. The steady states are $10/0.15 = 66.7$ tonnes and, with loading halved, $5/0.15 = 33.3$ tonnes. From $P(0) = 50$ with $L = 5$, the time to reach 45 tonnes is

$$t = -\frac{1}{0.15} \ln \frac{45 - 33.3}{50 - 33.3} \approx 2.38 \text{ years},$$

in line with the 2.35 years reported from the run (yearly saved points are interpolated; Euler integration adds a small error).

Under the answer, suggestion buttons such as **Open Scenarios** or **Open Sensitivity** open the

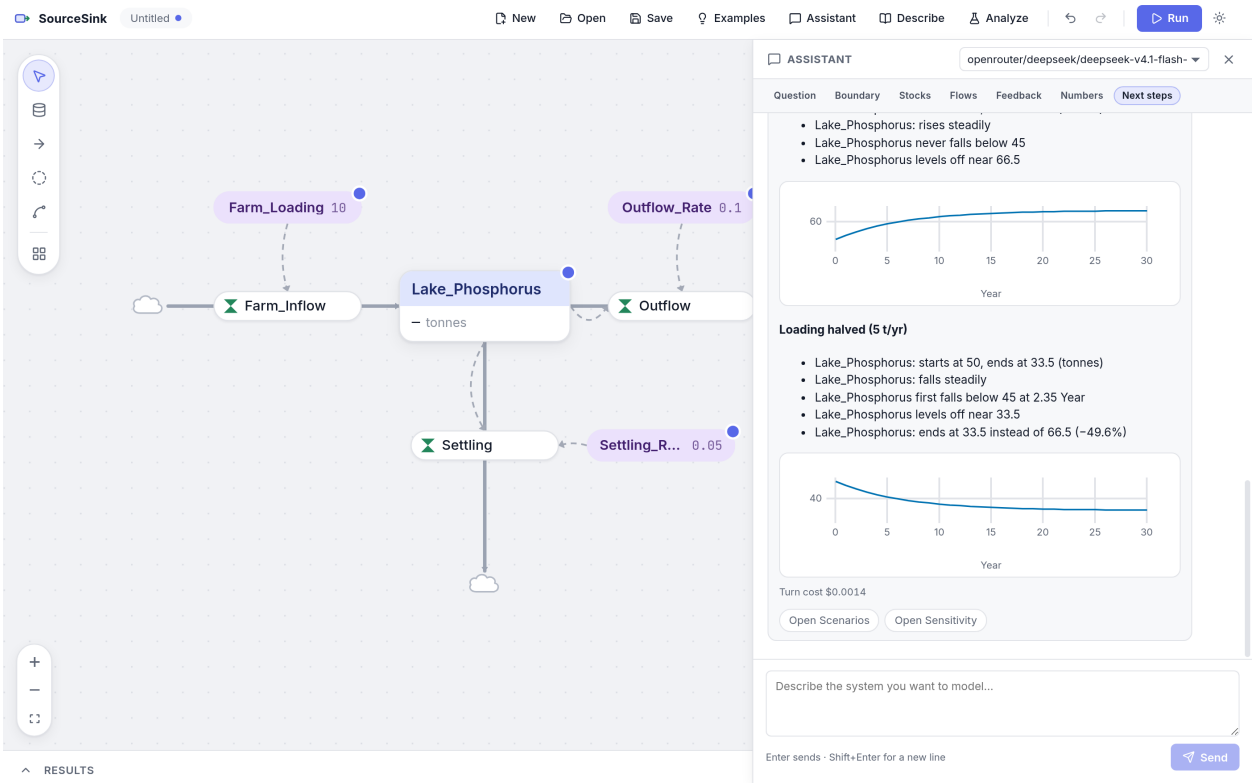


Figure 6.2: A grounded answer. With loading halved, phosphorus falls below 45 tonnes after 2.35 years and levels off near 33.5 tonnes, about half the base case.

relevant panel already filled in with the parameters from the conversation.

CHAPTER 7

Tutorial 6: Tidy a crowded diagram

Diagrams grow. Elements added by hand, imported from older files or proposed by the assistant can end up on top of each other. SourceSink can re-arrange the whole diagram for you.

Step by step

1. Open a crowded model. When shapes overlap, a small pill at the top of the canvas offers **Tidy up**.
2. Click **Tidy up** in the pill, or the grid button at the bottom of the palette, or press L.
3. If you prefer the old arrangement, press Ctrl+Z: tidying is a single undoable step.

The layout follows the usual conventions of stock-and-flow diagrams. Each chain of stocks connected by flows gets its own row, ordered in the direction material moves. Flows to and from the outside use the free ends of the row first and then fan out above (inflows) or below (outflows) the stock. Flows that run backwards, such as a recycle, loop below the row. Converters sit next to the element they feed.

7.1 Opening legacy models

Click **Open** and choose a `.stm` or `.itm` file. SourceSink reads the model, its equations, graphical functions, arrays and diagram positions, and reports in a notice anything it could not import faithfully. The original arrangement is kept and spread out to suit SourceSink's larger shapes; use **Tidy up** if you prefer a fresh layout. Save the result as XMILE to keep working with it.

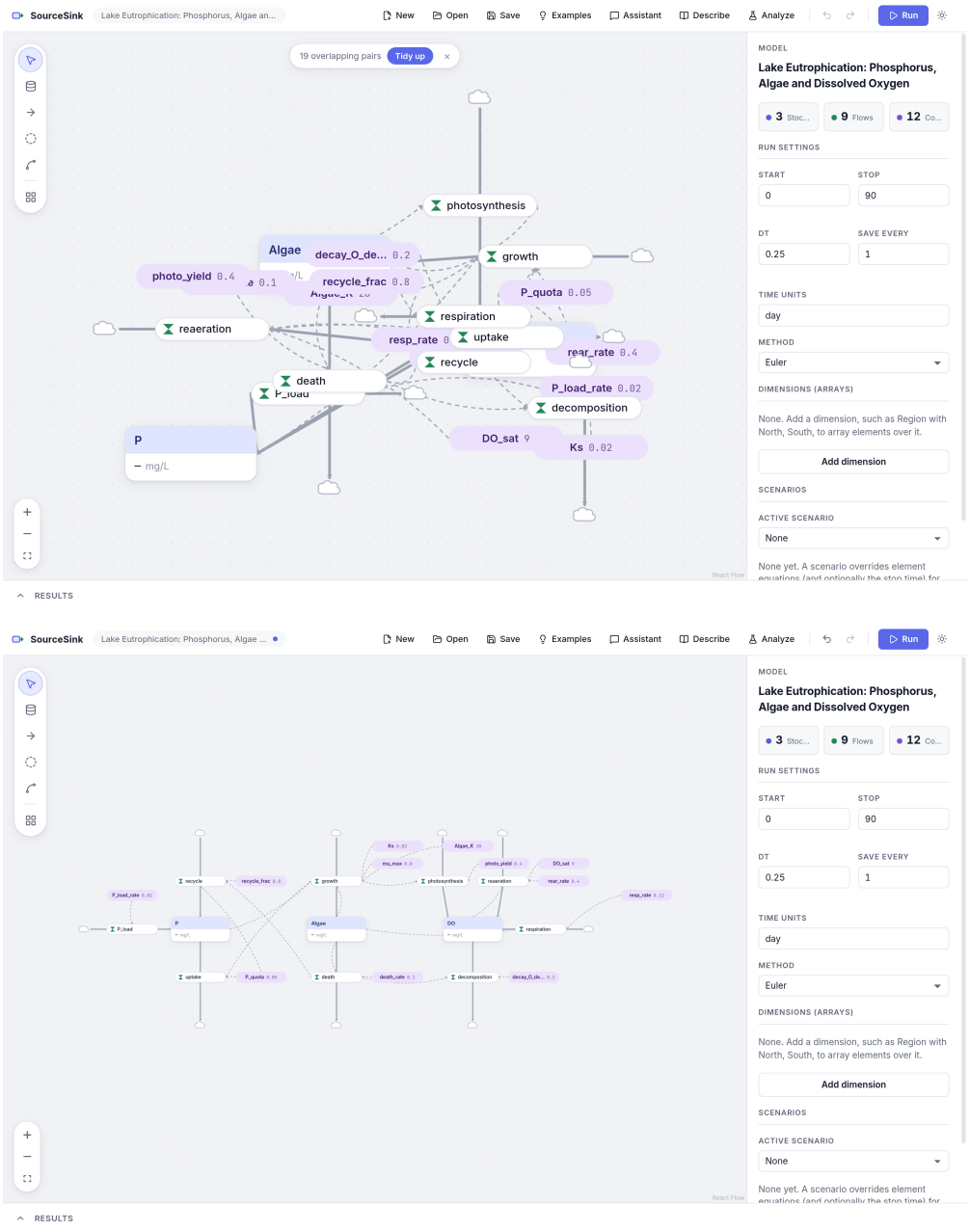


Figure 7.1: A lake eutrophication model before (top) and after (bottom) **Tidy up**.

CHAPTER 8

Tutorial 7: The command line

Everything you have done in the app can be scripted. The `sourcesink` command runs models, sweeps parameters, fits data and runs sensitivity analyses, and writes CSV or JSON for other tools. From a copy of the repository, use `npx tsx src/cli/src/run.ts` in place of `sourcesink`.

```
# Run a model and print selected series as CSV
sourcesink run examples/environmental/streeter-phelps.xmile --columns BOD,DO

# Check units
sourcesink units examples/environmental/streeter-phelps.xmile

# Describe the model in plain language
sourcesink explain examples/environmental/streeter-phelps.xmile

# Monte Carlo with Latin hypercube sampling
sourcesink montecarlo examples/environmental/streeter-phelps.xmile \
  --param kd=0.2:0.5 --runs 200 --lhs --columns DO_deficit --percentiles 5,50,95

# Morris screening of the deficit peak
sourcesink sensitivity examples/environmental/streeter-phelps.xmile \
  --param kd=0.2:0.5 --param ka=0.4:0.8 --metric DO_deficit:max
```

The complete reference for every command and option, the model file format and the MCP server (which lets AI agents build and analyze models through SourceSink) is in `docs/llm/cli-guide.md`.

CHAPTER 9

Reference

9.1 Elements and their options

Stocks. Equation = initial value. **Non-negative** keeps the stock at or above zero by limiting its outflows. A stock can also be a *conveyor* (material rides for a transit time, optionally with a capacity and leakage), a *queue* (first in, first out) or an *oven* (fills, cooks for a time, empties), the discrete-event stocks used for queues, pipelines and batch processes.

Flows. **Uniflow** prevents negative values; **Biflow** allows the flow to run both ways. A flow out of a conveyor can be a *leak*.

Converters. A constant or an equation. A converter can also be a graphical function: a lookup table of points with linear, held (step) or discrete interpolation.

Arrays. Under **Dimensions** (nothing selected), define a dimension such as `Region` with elements `North, South`. Any element can then be arrayed over it, with one equation for all elements or one per element.

9.2 Integration methods

Method	**Update for $\dot{S} = f(t, S)$ with step h**
Euler	$S_{n+1} = S_n + hf(t_n, S_n)$. First order: error $\propto h$.
Runge–Kutta 2 (Heun)	$k_1 = f(t_n, S_n)$, $k_2 = f(t_n + h, S_n + hk_1)$, $S_{n+1} = S_n + \frac{h}{2}(k_1 + k_2)$.
Runge–Kutta 4	$S_{n+1} = S_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ with the classic four stages. Error $\propto h^4$.
Adaptive (RK45)	Dormand–Prince pair: estimates its own error each step and shrinks or grows the step to meet a tolerance. Best for stiff or fast models.

9.3 Analysis methods at a glance

Question	Tool	Output
What values fit my data?	Calibrate	Estimates $\pm$ standard errors, RMSE, NSE, R^2, bias, identifiability warnings
Which parameters matter?	Morris	μ^* (importance) and σ (interaction) per parameter
How much of the variance?	Sobol	S_1 and S_T with confidence intervals
How wide are the outcomes?	Uncertainty	Median and percentile bands over time
What if a value changes?	Scenarios	A separate run with overridden equations

9.4 Themes

The sun/moon button at the top right switches between light, dark and "follow the system." Figure 9.1 shows the dark theme.

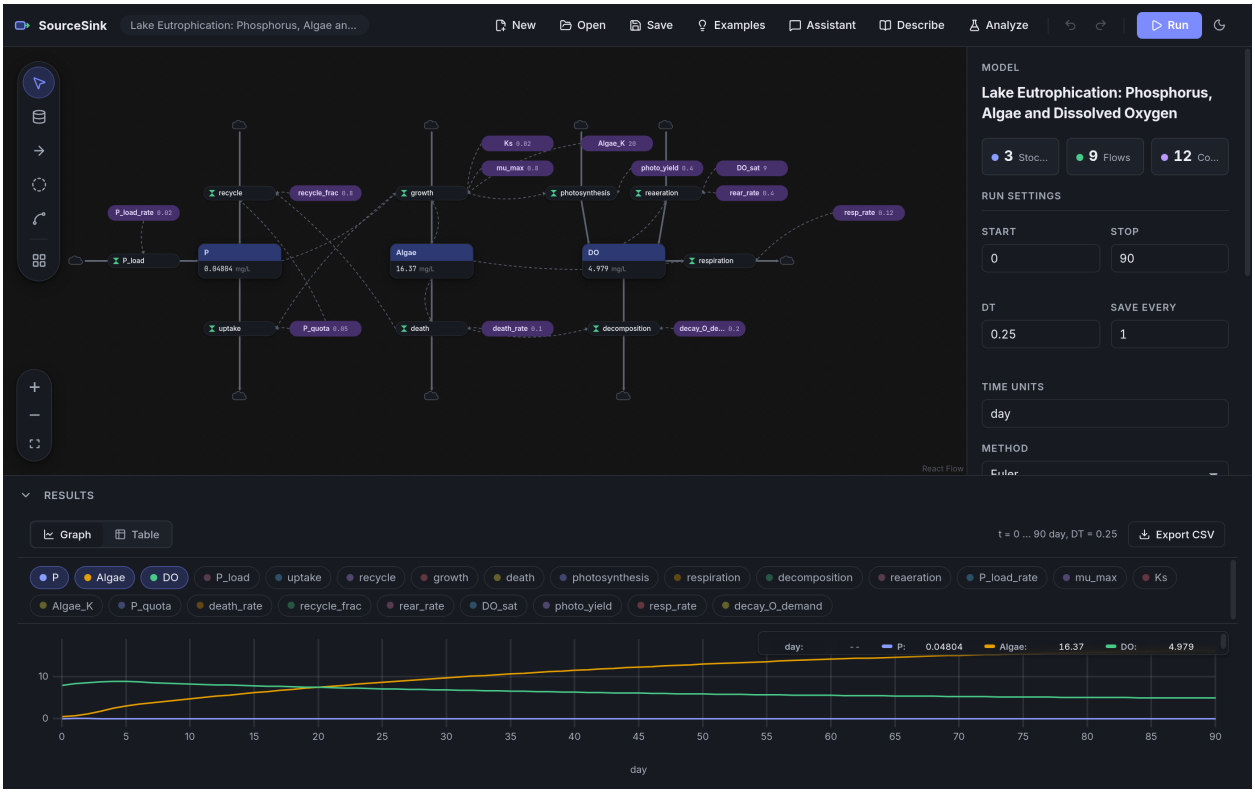


Figure 9.1: The dark theme with the lake eutrophication example after a run.

APPENDIX A

Equation functions

Equations use +, −, *, /, ^, comparisons (<, <=, >, >=, =, <>), AND, OR, NOT and IF *condition* THEN *a* ELSE *b*. TIME is the current time and DT the time step. Names are case-insensitive.

Function	Meaning
Mathematics	
ABS(x), SQRT(x), EXP(x)	absolute value, square root, e^x
LN(x), LOG10(x)	natural and base-10 logarithm
SIN, COS, TAN, ARCTAN	trigonometry in radians
MIN(a,b,…), MAX(a,b,…)	smallest and largest argument
MOD(a,b), INT(x), ROUND(x)	remainder, floor (INT(−8.9) = −9), nearest integer
POWER(a,b)	a^b
SUM, MEAN, STDDEV	sum, mean and population standard deviation of the arguments
Time inputs	
STEP(h, t)	0 before time t, h from t on
RAMP(slope, start, end)	rises at *slope* from *start* until *end*
PULSE(v, first, interval)	moves volume v in one DT, repeating every *interval*
SINWAVE(a, p), COSWAVE(a, p)	periodic inputs of amplitude a and period p
Delays and smoothing	
DELAY1(in, d), DELAY3(in, d)	first- and third-order material delays
DELAYN(in, d, init, n)	nth-order material delay
DELAY(in, d)	pipeline delay: the input as it was d time units ago
SMTH1(in, t), SMTH3(in, t), SMTHN(in, t, n)	exponential smoothing (first, third, nth order)
TREND(in, t), FORCST(in, t, h)	fractional trend; extrapolation h time units ahead
Random numbers (repeatable with a seed)	
RANDOM(min, max), NORMAL(m, s)	uniform and normal random values
EXPRND(mean), POISSON(mu)	exponential and Poisson random values
MONTECARLO(p)	1 with probability p% per time unit
Finance	
PV, FV, PMT, NPV	present value, future value, payment, net present value
CGROWTH(pct)	DT-independent compound growth rate
Discrete stocks and cycle times	
QLEN, QELEM, CAP	queue length, element of a queue or conveyor, capacity
TRANSTIME, COOKTIME, OSTATE	conveyor transit time, oven cook time and state
CTMEAN, CTMIN, CTMAX, CTSTDDEV, CYCLETIME, THROUGHPUT	cycle-time statistics
Other	
PREVIOUS(x), INIT(x)	value one DT ago; value at the start

Function	Meaning
`COUNTER(a, b), SWITCH(a, b), PCT(x)`	counter; 1 if $a > b$; $100x$
`DERIVN(x, n)`	nth time derivative by backward differences

APPENDIX B

Units

Write units the way you would on paper: `mg/L`, `1/day`, `m3/s` (a digit after a letter is an exponent), `m^3`, `kg m-2`, `tonnes/year`, `people`, `%`, `dimensionless`. Time units are recognized in their common forms (s, min, h, day, week, month, year) and convert to one another; masses (mg, g, kg, tonne), lengths (mm, cm, m, km) and volumes (mL, L, m3) also convert. Any other word, such as `fish` or `dollars`, is its own unit.

APPENDIX C

Keyboard shortcuts

V	Select tool	Ctrl + S	Save
S	Stock tool	Ctrl + Shift + S	Save as
F	Flow tool	Ctrl + O	Open
C	Converter tool	Ctrl + Z	Undo
K	Connector tool	Ctrl + Y	Redo
L	Tidy up the layout	Ctrl + Enter	Run
Esc	Select tool, clear selection	Delete	Delete the selection

APPENDIX D

Troubleshooting

A stock shoots to enormous values or NaN.
The model is probably stiff for the chosen DT. Halve DT, or choose **Adaptive (RK45)**. Check that a rate is not in the wrong time units (a unit warning usually says so).

A stock goes negative and should not.
Tick **Non-negative** on the stock, or make the outflow depend on the stock so it shrinks as the stock empties.

An equation says a name is unknown.
Check the spelling, and connect the element with the connector tool (K); the inspector offers to add the missing connector.

Calibration does not converge.
Narrow the parameter ranges, use **log** for parameters spanning orders of magnitude, add restarts, or try differential evolution. Read the identifiability warnings: two parameters that always move together cannot both be estimated from these data.

The assistant is unavailable.
Install OpenCode, run `opencode providers` to sign in to OpenRouter, then click **Check again** in the panel.

References

[1] Bass, F. M. (1969). A new product growth for model consumer durables. *Management Science*, 15(5), 215–227. https://doi.org/10.1287/mnsc.15.5.215

[2] Harte, J. (1988). *Consider a Spherical Cow: A Course in Environmental Problem Solving*. University Science Books.

[3] Morris, M. D. (1991). Factorial sampling plans for preliminary computational experiments. *Technometrics*, 33(2), 161–174. https://doi.org/10.1080/00401706.1991.10484804

[4] Saltelli, A., Annoni, P., Azzini, I., Campolongo, F., Ratto, M., & Tarantola, S. (2010). Variance based sensitivity analysis of model output. Design and estimator for the total sensitivity index. *Computer Physics Communications*, 181(2), 259–270. https://doi.org/10.1016/j.cpc.2009.09.018

[5] Streeter, H. W., & Phelps, E. B. (1925). A study of the pollution and natural purification of the Ohio River. III. Factors concerned in the phenomena of oxidation and reaeration. Public Health Bulletin No. 146, U.S. Public Health Service.

[6] Vollenweider, R. A. (1975). Input-output models with special reference to the phosphorus loading concept in limnology. *Schweizerische Zeitschrift für Hydrologie*, 37(1), 53–84. https://doi.org/10.1007/BF02505178